



Generative AI Academy

Today's Agenda

- Recap from EDA of structured data
- Microsoft Azure Services for EDA
- Quiz
- EDA of unstructured Data
 - Text
 - Images
- Assignments and Module End Project

EDA Recap – t-test

A t-test is typically used to compare means of (usually) **normally distributed continuous data**.

Below are common types of t-tests

One-sample t-test

Compares the mean of a single sample to a known or hypothesized population mean.

Two-sample (independent) t-test

Compares the means of two independent groups (e.g., control group vs. treatment group).

Paired t-test

Compares the means of paired observations (e.g., the same individuals measured before and after an intervention).

EDA Recap – chi-squared test

A chi-squared test is typically used for categorical data.

Key scenarios include:

Chi-Square Goodness-of-Fit

You have a single categorical variable and want to see if the distribution of categories differs from an expected distribution.

Chi-Square Test of Independence

You have two categorical variables (e.g., gender and preference for a certain product), and you want to check whether they are independent or associated.?)

EDA Recap

Scenario: You ask 100 people their preferred type of cuisine (Italian, Chinese, Indian, Mexican).

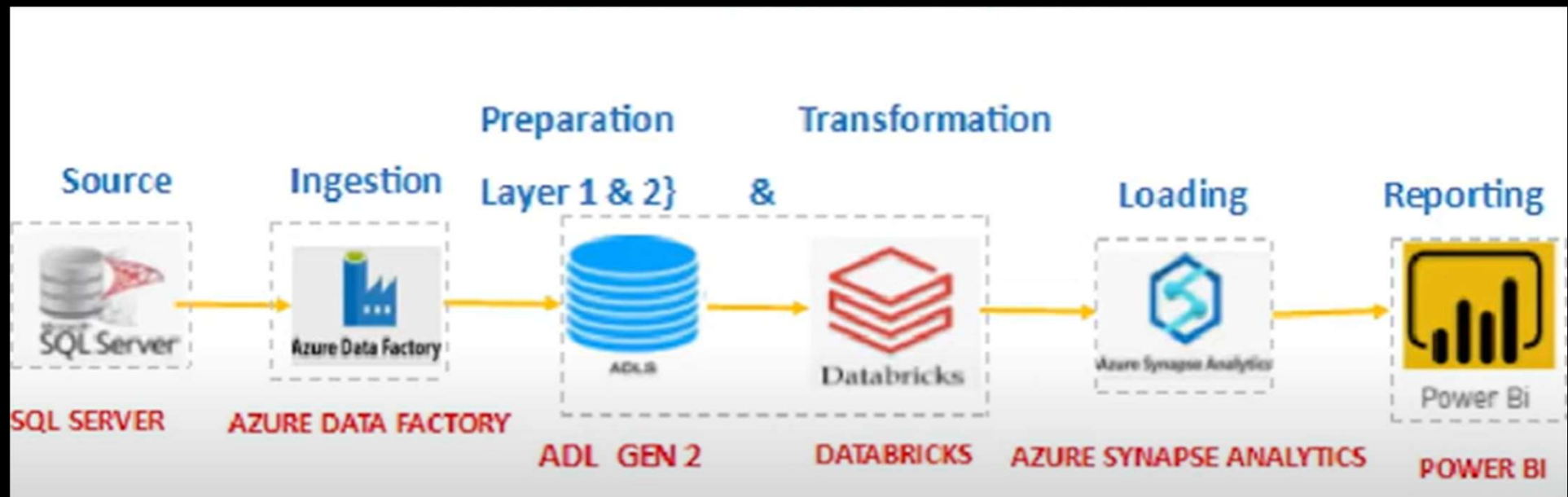
You expect (based on some prior assumption) that 25% prefer each cuisine. You record the actual counts and see if the real preferences differ significantly from the 25%-25%-25%-25% assumption.

In Python how you would go about doing the hypothesis testing for this?

What is the null hypothesis?

Which test can be used? (t-test? Chi-squared?)

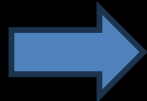
Azure Services for EDA



Azure Services for EDA

Data ingestion

- **Azure Data Factory**
Low code/no code tool:
Visual pipeline designer, control flows, event triggers, scheduling
- Data ingestion, ETL at scale
- Integrates with Databricks, Synapse
- For async http calls, Python asyncio, aiohttp are suitable



Data Eng., model training

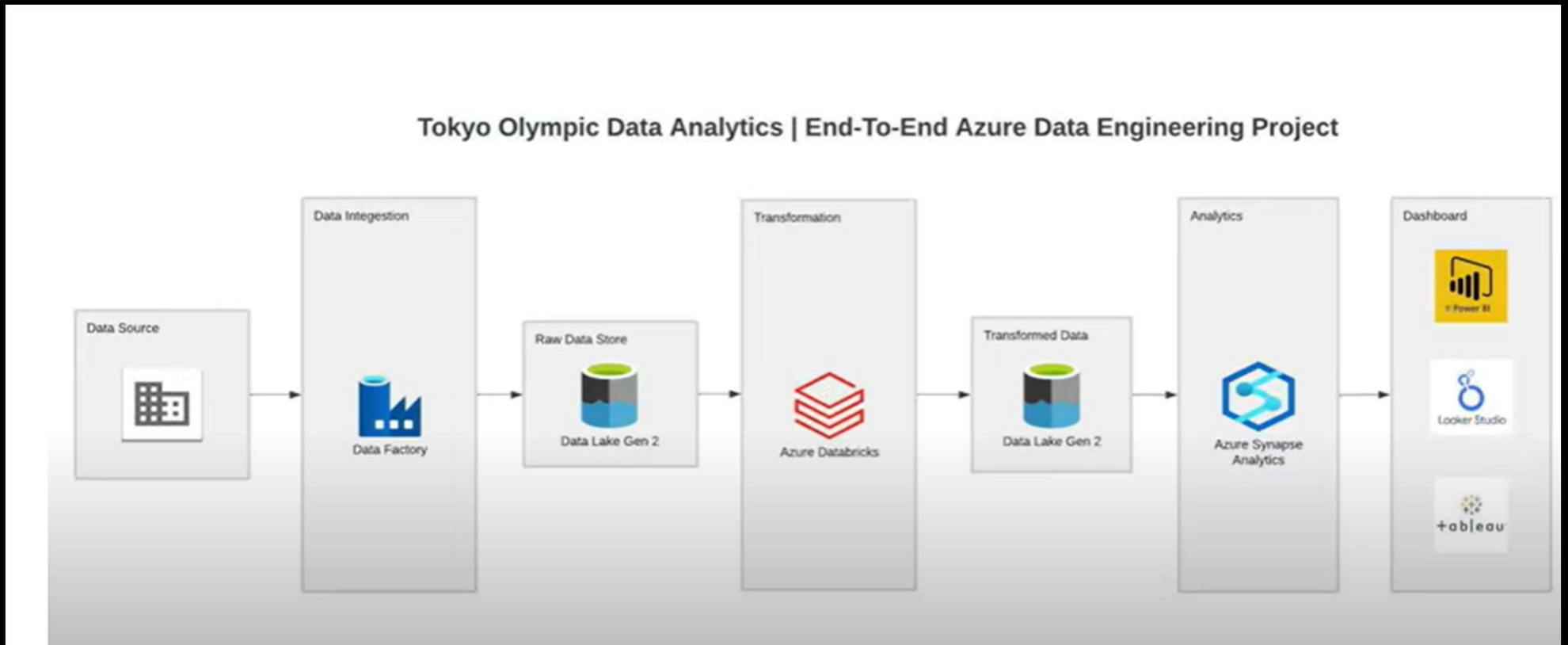
- **Azure Databricks**
data cleaning, feature engineering at scale with Apache Spark
- Distributed Model training with Apache Spark. Integrates with Hadoop
- Notebooks
- Metadata, Delta Tables



Data Storage

- Parquet files in Azure Data Lake or AWS S3
- Azure Synapse
If you require complex sql queries, reporting or ad-hoc analysis on structured data

Azure Services for EDA – real life example



<https://www.youtube.com/watch?v=laA9YNlg5hM>

QUIZ TIME!!

EDA of unstructured data:

Text

Text Processing with Python

```
with open('data/numbers.txt', 'w') as fp:  
    fp.write('2+2')  
    fp.write('\n')  
    fp.write(2+2.45)
```

What will the above code do?

Text Processing with Python

```
with open('data/numbers.txt', 'r') as fp:  
    text = fp.read().splitlines()  
    print(text)  
text
```

data/numbers.txt

```
2+2  
4.45
```

What will above code print?

How to make it process the mathematical expressions?

```
with open('data/numbers.txt', 'r') as fp:  
    text = fp.read().splitlines()  
    text = [eval(x) for x in text]  
text
```

Natural Language Processing (NLP)

Linguistic Basics

1. Lemmatization
2. Stemming
3. Part of Speech Tagging

Text Identification/ Extraction

4. Stop Words
5. Pattern Matching
6. Sentence Segmentation
7. Named Entity Recognition

Text Representation

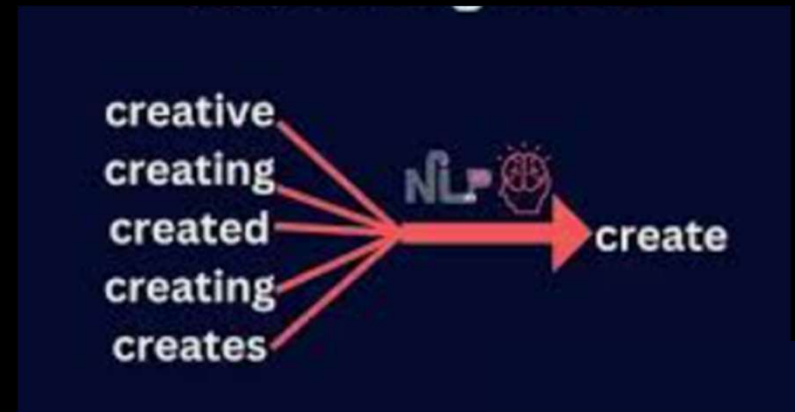
8. Tokenization
9. Word Embedding
10. Bag-of-words/TF-IDF

NLP - Stemming

Obtain the base or root form of words by removing letters from the word's end

```
from nltk.stem.porter import *  
  
p_stemmer = PorterStemmer()  
words = ["runner", "running", "ran"]  
for word in words:  
    print(word+' --> '+p_stemmer.stem(word))
```

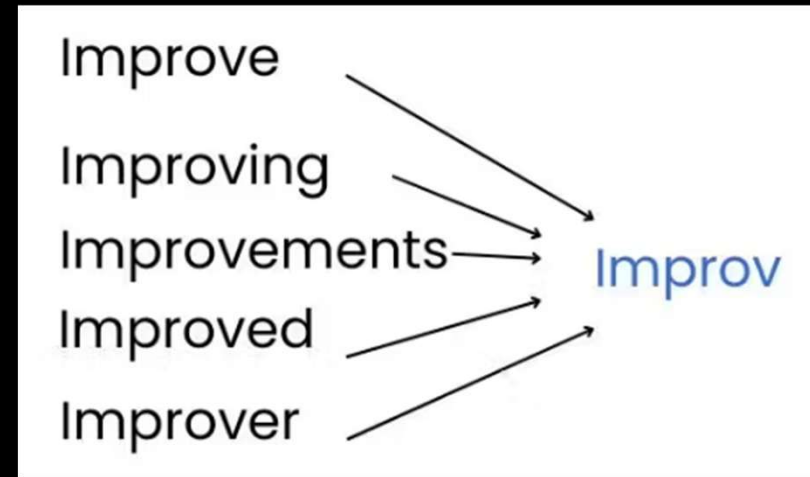
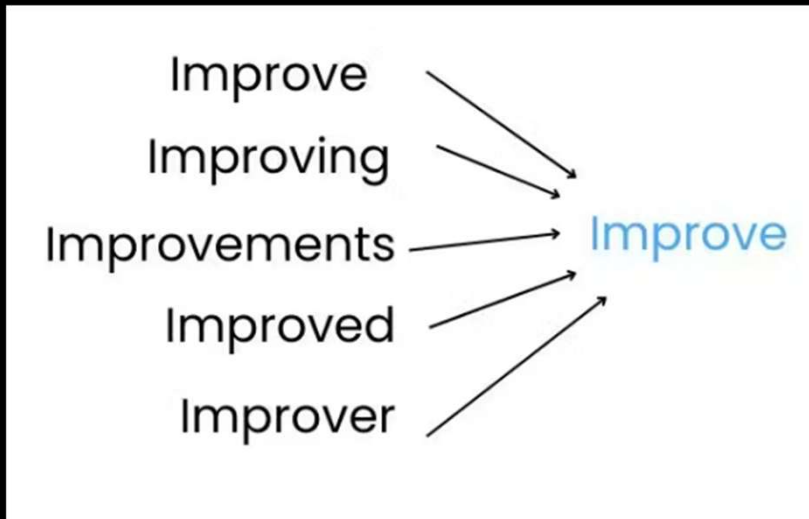
```
runner --> runner  
running --> run  
ran --> ran
```



NLP - Lemmatization

lemmatization is a more sophisticated linguistic process that aims to reduce a word to its base or dictionary form

Lemmatization takes into account factors such as part-of-speech (POS) tags and contextual understanding to ensure accurate and meaningful transformations.



Vs Stemming

Parts Of Speech (POS) Tagging

Part of speech refers to the grammatical category of a word in a sentence, such as noun, verb, adjective, adverb, pronoun, preposition, conjunction, or interjection.

Used to identify named entities and speech recognition

Stop words

Common words such as “a” and “the” occur so frequently in text that they often do not carry significant meaning compared to nouns, verbs, and modifiers.

Spacy provides a built-in list of approximately 305 English stop words that can be readily utilized.

Named Entity Recognition (NER)

Named Entity Recognition (NER) entails the identification and classification of named entities present in the given text.

Named entities represent recognizable entities : individuals, organizations, locations, dates, numerical expressions, and others.

The screenshot displays a web-based NER interface. At the top, a blue header bar contains seven labels with corresponding single-letter codes: 'Person' (p), 'Loc' (l), 'Org' (o), 'Event' (e), 'Date' (d), and 'Other' (z). Below this, a text snippet about Barack Obama is shown. The text is: 'Barack Hussein Obama II (born August 4, 1961) is an American attorney and politician who served as the 44th President of the United States from January 20, 2009, to January 20, 2017. A member of the Democratic Party, he was the first African American to serve as president. He was previously a United States Senator from Illinois and a member of the Illinois State Senate.' Each entity in the text is enclosed in a colored box with a small 'x' icon in the top right corner. The labels used are: 'Person' (blue), 'Date' (red), 'Other' (purple), 'Loc' (yellow), 'Org' (dark grey), and 'Event' (green).

Entity	Label	Code
Barack Hussein Obama II	Person	p
August 4, 1961	Date	d
American	Other	z
the United States	Loc	l
January 20, 2009	Date	d
January 20, 2017	Date	d
Democratic Party	Org	o
African American	Other	z
United States Senator	Other	z
Illinois	Loc	l
Illinois State Senate	Org	o

<https://demos.explosion.ai/displacy-ent>

Text representation - tokenization

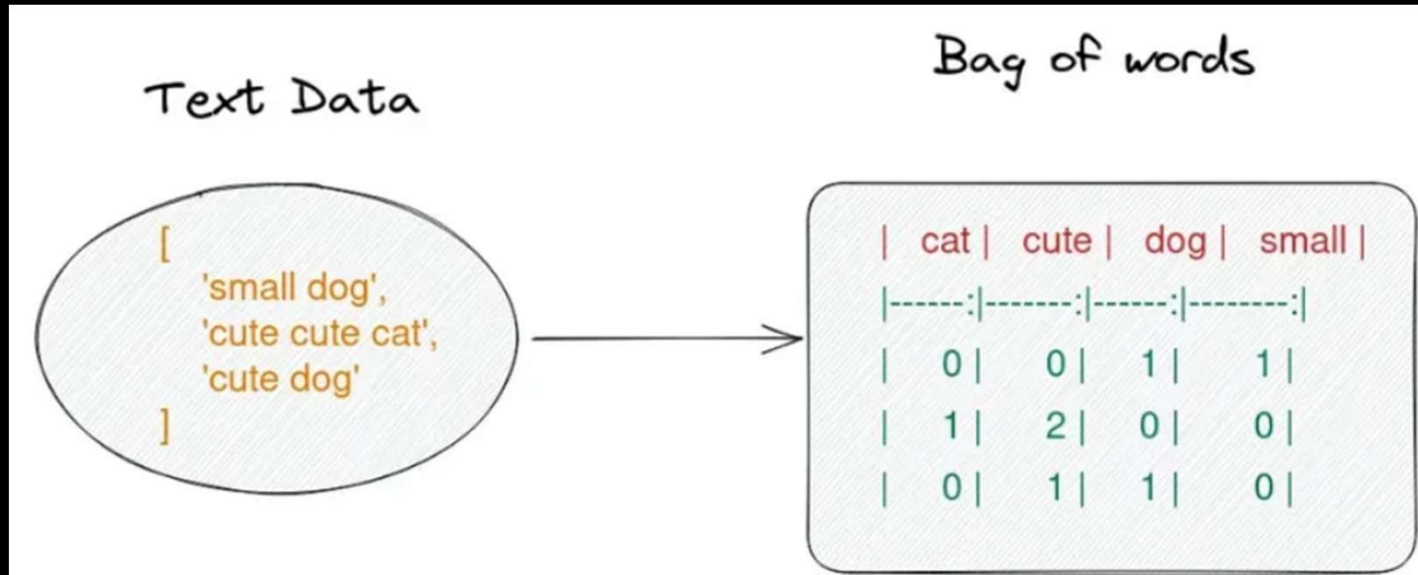
Tokenization involves breaking down the original text into smaller components known as tokens. These tokens can be created based on contiguous sequences of characters or words.

N-gram refers to a consecutive sequence of n items, where an item can be a character or a word.

<https://platform.openai.com/tokenizer>

Determines the vocabulary of the model

Text representation – Bag of Words



Converts each word into a vector based on the frequency of the text, without considering order

Text representation – TF-IDF

Term Frequency (TF) / Inverse Document Frequency (IDF)

Term Frequency (TF)

This measures how frequently a term (word) appears in a document. It assigns a higher weight to words that occur more frequently within the document.

Inverse Document Frequency (IDF)

This part measures how unique or rare a term is across all documents. It assigns a higher weight to words that appear less frequently across the corpus but provide more unique or informative content within a document.

Text representation – TF-IDF

Term Frequency (TF) / Inverse Document Frequency (IDF)

Corpus D		
$+1 \rightarrow$	d_1 A quick brown fox jumps over the lazy dog. What a fox!	$TF-IDF = 0.17 \times 0 = 0$ (“fox”, d_1 , D)
$+1 \rightarrow$	d_2 A quick brown fox jumps over the lazy fox. What a fox!	$TF-IDF = 0.25 \times 0 = 0$ (“fox”, d_2 , D)

Answer: Using TF-IDF, the word “fox” is equally relevant for both document d_1 and document d_2

Question: How word fox is relevant to corpus D documents?

Solution:

TF-IDF

TF is the frequency of any “term” in a given “document”.

IDF is constant per corpus, and accounts for the ratio of documents that include that specific “term”.

$$TF(\text{“fox”}, d_1) = 2 / 12 = 0.17$$

$$TF(\text{“fox”}, d_2) = 3 / 12 = 0.25$$

$$IDF(\text{“fox”}, D) = \log(2/2) = 0$$

Limitations of BoW and TF/IDF

BoW and TF-IDF are traditional text representation techniques, they have major limitations:

No Semantic Meaning, cannot capture similarity

Represent text as sparse vectors where each word is a separate dimension.

For example, "king" and "queen" have no relationship in BoW/TF-IDF.

High Dimensionality (Sparse Representation)

If a vocabulary has 100,000 words, each document is a 100,000-dimensional vector, making computations inefficient.

No Context Awareness

BoW & TF-IDF treat all occurrences of a word the same, regardless of sentence structure.

No Word Order Information

BoW and TF-IDF ignore word order, so "I love cats" and "Cats love me" have the same representation.

Word2vec (Google 2013)

Solves many of the issues with previous approaches

Word2Vec is a

- Neural network-based model for learning word embeddings
- Introduces word embeddings: dense vector representations of words that capture their semantic meaning based on the context they appear in.
- It transforms words into high-dimensional numerical vectors, allowing words with similar meanings to have similar vector representations.

A major breakthrough in NLP

How Word2vec (Google 2013) solves these issues?

Dense Vector Representations

Represents words as low-dimensional dense vectors
(e.g., 300 dimensions instead of 100,000).

Captures Semantic Meaning

Similar words have similar vector representations:

king - man + woman $\approx$ queen

Learns Context from Word Usage

Uses CBOW (Continuous Bag of Words) and Skip-gram to learn relationships based on nearby words.

Handles Synonyms and Similarity

Places similar words close in vector space, making it useful for NLP tasks.

A major breakthrough in NLP

How Word2vec learns Symantec meaning and relationship?

Self supervised learning

Skip-gram Model

Given a word, it predicts surrounding words.

Example: From "The cat sits on the mat," the model learns that "cat" is often surrounded by words like "sits," "on," and "mat."

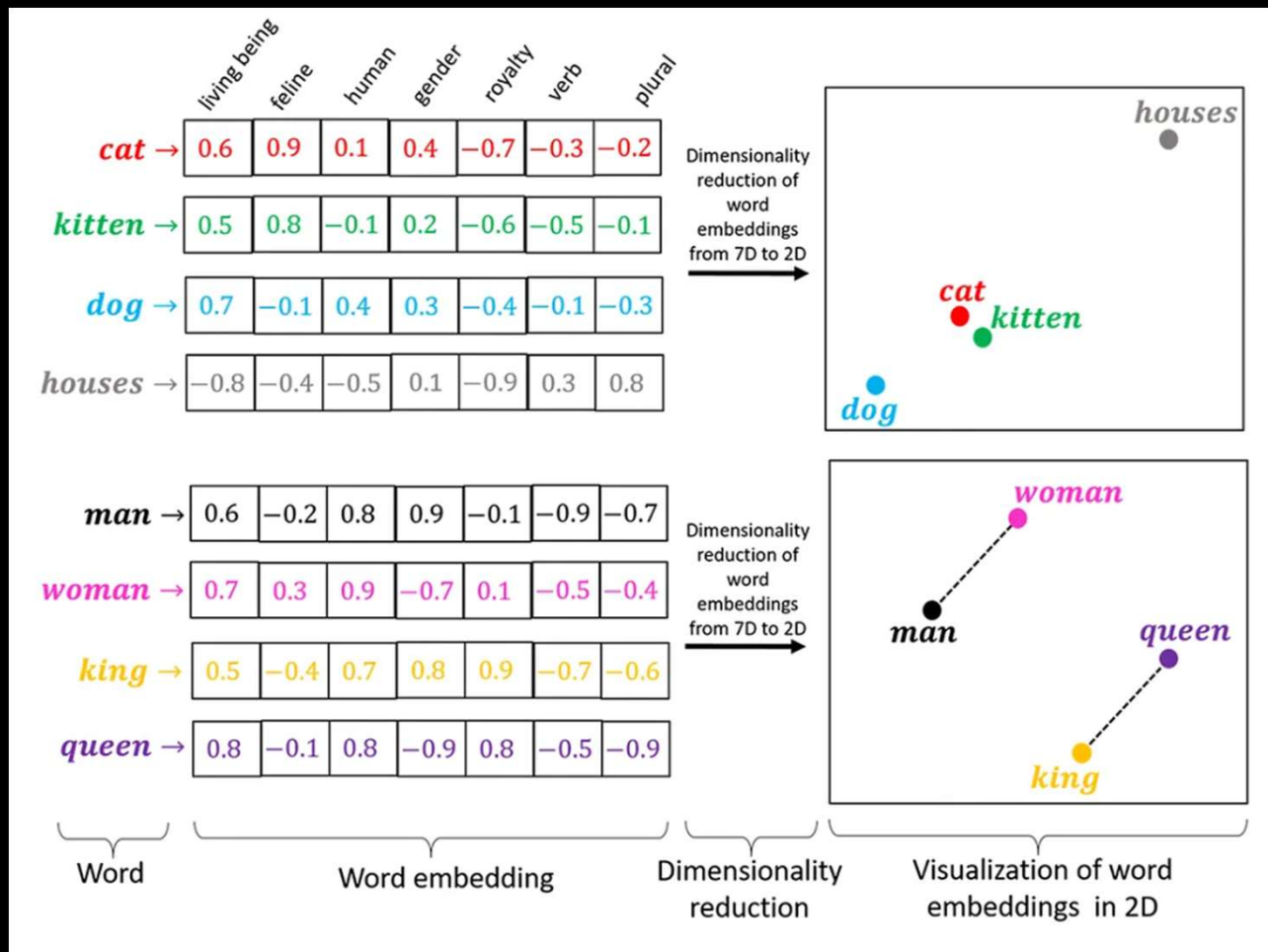
This helps capture the semantic meaning of "cat."

Continuous Bag of Words (CBOW) Model

Given surrounding words, it predicts the missing word.

Example: Given "The ____ sits on the mat," the model predicts "cat."

Vectors and Embeddings



Video: Vectors and Embeddings

Popular Python libraries for NLP

NLTK (Natural Language Toolkit)

Creator: Steven Bird, UPenn

Year of Creation: 2001

- Academic focus
- Includes tokenizers, stemmers, corpora etc. but is more of a toolkit

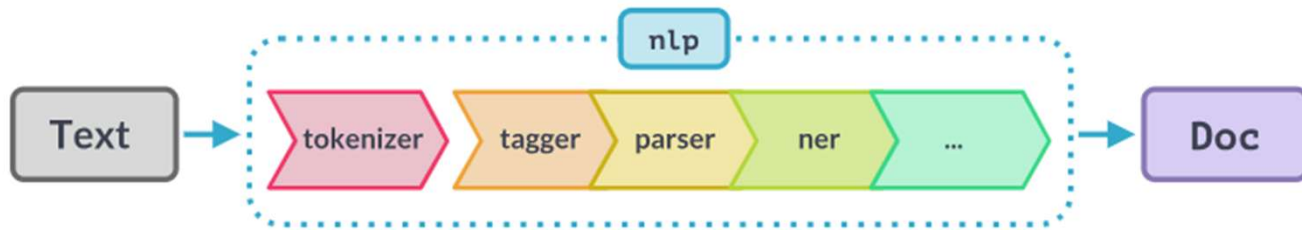
spaCy

Creator: Matthew Honnibal, Germany

Year of Creation: 2015

- Designed for industrial use
- Comes with tokenizers, PoS tagging, NER, dependency parsers
- Pre-trained models for many languages

spaCy pipeline



<https://spacy.io/usage/processing-pipelines>

spaCy pipeline

NAME	COMPONENT	CREATES	DESCRIPTION
tokenizer	Tokenizer 	Doc	Segment text into tokens.
PROCESSING PIPELINE			
tagger	Tagger 	Token.tag	Assign part-of-speech tags.
parser	DependencyParser 	Token.head , Token.dep , Doc.sents , Doc.noun_chunks	Assign dependency labels.
ner	EntityRecognizer 	Doc.ents , Token.ent_iob , Token.ent_type	Detect and label named entities.
lemmatizer	Lemmatizer 	Token.lemma	Assign base forms.
textcat	TextCategorizer 	Doc.cats	Assign document labels.

How does spaCy do NER?

spaCy uses pre-annotated datasets and trains a neural network to be able to infer named entities

Example labeled sentence:

```
rust
```

```
"Apple" -> ORG (Organization)
```

```
"New York" -> GPE (Geopolitical Entity)
```

```
"Elon Musk" -> PERSON
```

Uses CNN to train

Inferencing:

```
"Apple is hiring in New York."
```

```
["Apple" -> ORG, "New York" -> GPE]
```

Training the word2vec model

CBOW (Continuous Bag Of Words)

Predicts center word from context

Skip-gram

Predicts context from the center word

TODO

Lda, Isi unsupervised

NLTK Vs Spacy

Diff bet topic modeling and NER

Spacy – embeddings are static or contextual?

When context is limited, does it mean it takes it as batches in input?

(Ans: no – it takes “all” input, context limit such as 8-10 tokens are internal)

Spacy customization: retraining or fine tuning?

<https://chatgpt.com/share/681b3ac6-983c-800f-9dea-8ee82037b48e>

Code walkthrough: Spacy

Topic Modeling

LSI (Latent Semantic Indexing)

A linear algebra-based method for dimensionality reduction in text data. It represents words and documents in a reduced vector space where similar documents and words are closer together.

LDA (Latent Dirichlet Allocation)

Words and documents in a reduced vector space where similar documents and words are closer together.

LSI and LDA for topic modeling

Feature	LSI (Latent Semantic Indexing)	LDA (Latent Dirichlet Allocation)
Approach	Algebraic (SVD)	Probabilistic (Bayesian Inference)
Interpretability	Harder (topics are abstract vectors)	Easier (topics are word distributions)
Topic Assumption	Documents have <i>one</i> dominant topic	Documents are mixtures of topics
Computational Cost	Faster	Slower (more parameters to estimate)
Handles Polysemy?	No	Yes
Scalability	Works well with medium-sized data	Works well with large datasets

Popular Python libraries for NLP topic modeling

Gensim

Focus: Topic modeling, document similarity, and vector space modeling.

Key Features:

- Efficient implementations of word2vec, doc2vec, and related algorithms.
- Scalable for large text corpora.
- Well-suited for tasks like text similarity, LSI, LDA topic modeling.

Code walkthrough: Gensim

Popular Python libraries for NLP contd.

TextBlob

Focus: Simpler, more beginner-friendly NLP tasks

Key Features:

- Easy-to-use API for sentiment analysis, part-of-speech tagging, noun phrase extraction, and more.
- Built on top of NLTK and Pattern, so it's not as fast as spaCy but is great for small projects or quick prototypes.

Others: scikit-learn, PyTorch, Tensorflow, Transformers
(to be covered in Deep learning module)

EDA of unstructured data:

Images

How are images represented?

- Matrix of pixels?
- A 512x512px image will be what size matrix?
If it has RGB, then $512 \times 512 \times 3$

EDA pipeline for image processing

1. **Load** a 512×512 image in RGB.
2. **Downsample** to 128×128 (if you want fewer pixels).
3. (Optional) **Gaussian Smoothing** on the 128×128 image.
4. **Flatten** that $128 \times 128 \times 3$ to $(128^2, 3)$ and apply **PCA** if you want to reduce color dimensions.

Each step addresses different aspects of efficiency or analysis:

- Downsampling → fewer pixels to process (efficiency).
- Smoothing → remove noise, high-frequency details (better feature extraction).
- PCA → reduce color/feature dimension (fewer channels, easier color-space analysis).

EDA pipeline for image processing - details

1. Resize / Decrease Resolution:

- Resize the image to a standard size (e.g., $512 \times 512 \rightarrow 256 \times 256$ or even smaller if needed).

2. Normalization / Standardization:

- Apply normalization (or standardization) to ensure the pixel values are on a consistent scale. This is especially important when working with algorithms that are sensitive to the scale of data.

3. Gaussian Filtering (Optional):

- Apply a Gaussian filter if the image contains noise or if you want to smooth out small, irrelevant details before compression.
- **Note:** This step is optional and context-dependent. If the image is already clean, you might skip this.

4. Dimensionality Reduction (SVD / PCA):

- Use SVD or PCA to decompose the image and keep only the top components (eigenvectors).
- This is the step where you achieve compression by reducing the number of values you need to store while still retaining the most important features of the image.

Gaussian as a smoothing kernel

1.2 Gaussian as a Smoothing Kernel

In **image processing**, the Gaussian kernel is frequently used as a **smoothing or blurring filter**. When you convolve an image I with a Gaussian kernel $G(x, y; \sigma)$, you produce a **blurred** or **smoothed** version of the image:

$$I_{\text{smoothed}}(x, y) = I(x, y) * G(x, y; \sigma),$$

where $*$ denotes the 2D convolution operation.

Why use Gaussian smoothing?

- It suppresses high-frequency details (noise, small textures) while retaining overall structure.
- It is naturally “well-behaved,” meaning it does not introduce sharp artifacts or ringing effects.
- In **scale-space theory**, the Gaussian function is key to systematically analyzing images at multiple scales (i.e., from coarse to fine).

Eigen Vectors

2.1 Eigenvectors and Eigenvalues

An **eigenvector** of a linear transformation T (represented by a matrix $\mathbf{A}$) is a vector $\mathbf{v}$ such that when $\mathbf{A}$ acts on $\mathbf{v}$, the vector is simply scaled by some scalar λ . Formally:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

- $\mathbf{v}$ is the **eigenvector**.
- λ is the **eigenvalue**.
- $\mathbf{A}$ is any square matrix (e.g., a 2×2 , 3×3 , ...).

Eigenvectors point in “principal directions” (i.e., directions along which the transformation acts by just stretching or compressing).

Eigen Vectors for dimensionality reduction in images

- In very simple terms, an eigenvector is a direction that doesn't get "twisted" by a certain transformation (like stretching, compressing, or rotating).
- Instead, along that special direction, the transformation just makes things bigger or smaller, without turning them.
- The eigenvalue is how much that direction is stretched or shrunk.
- Dimensionality reduction: Many algorithms (e.g., Principal Component Analysis, PCA) use eigenvalues and eigenvectors of a covariance matrix to find directions in data of maximum variance, reducing large dimensional data (like images) to fewer dimensions.

Image standardization

- Image standardization is the process of transforming images so that they have a consistent format, scale, and statistical properties.
- This is an important pre-processing step in many computer vision and machine learning pipelines because it helps algorithms work more reliably by ensuring that variations in illumination, contrast, or size do not negatively impact the analysis.

Image standardization

What It Does:

Transforms data so that it has a mean of 0 and a standard deviation of 1 (often referred to as a z-score transformation).

How It's Done:

For each pixel (or feature) in the image, you subtract the mean and divide by the standard deviation:

Purpose:

Centers the data and scales it based on variability.

Helps algorithms that assume the data is centered around zero or that are sensitive to the variance of the data.

Image Normalization

What It Does:

Rescales data so that the values fall within a specific range, typically $[0, 1]$ or sometimes $[-1, 1]$.

How It's Done:

For example, with min-max normalization for an image, you subtract the minimum pixel value and then divide by the range (maximum minus minimum):

Purpose:

Brings different images or features to a common scale.

Useful when the scale of the data matters (e.g., when using certain activation functions in neural networks).

Image compression

- Singular Vector Decomposition
- EigenFaces